

Apprendimento per Rinforzo su CarRacing-v3

Confronto tra DQN, DDQN e PPO

Leonardo Baccocchi, Nicholas Avena

Anno Accademico 2025/2026

Sommario

Questo elaborato presenta un confronto sperimentale tra quattro implementazioni di algoritmi di Reinforcement Learning applicati all'ambiente di simulazione CarRacing-v3 (Gymnasium): Deep Q-Network (DQN), Double DQN (DDQN), PPO con implementazione custom e PPO tramite Stable-Baselines3. L'obiettivo è valutare le differenze in termini di reward medio, stabilità e velocità di convergenza, traendo conclusioni sull'applicabilità di ciascun approccio in contesti di controllo veicolare.

Indice

1	Descrizione del Problema	3
1.1	Contesto: ADAS e Guida Autonoma	3
1.2	L'Ambiente CarRacing-v3	4
1.3	Formulazione come Processo Decisionale di Markov	5
2	Fondamenti Teorici	6
2.1	Q-Learning e Deep Q-Network (DQN)	6
2.1.1	Q-Learning	6
2.1.2	Deep Q-Network (DQN)	6
2.1.3	Double DQN (DDQN)	8
2.2	Proximal Policy Optimization (PPO)	8
2.2.1	Funzione di Vantaggio e GAE	8
2.2.2	Clipped Surrogate Loss	9
2.2.3	Loss Totale Actor-Critic	9
2.2.4	Azioni Discrete e Distribuzione Categorical	10
2.3	Confronto tra i Paradigmi	10

3	Implementazione	10
3.1	Pipeline di Preprocessing	10
3.2	Architettura della Rete Neurale	11
3.3	Evoluzione Iterativa di DQN e DDQN	11
3.3.1	Prima Iterazione: Baseline e Stabilizzazione	12
3.3.2	Secondo Miglioramento: Terzo Strato Convoluzionale	12
3.3.3	Terzo Miglioramento: Prioritized Experience Replay	12
3.3.4	Quarto Esperimento: Spazio delle Azioni Esteso a 9 Comandi	13
3.3.5	Versione Finale	13
3.4	PPO con Implementazione Custom	14
3.5	SB3-PPO con Ambienti Paralleli	15
4	Risultati e Analisi	16
4.1	Ablation Study: Evoluzione delle Varianti DQN/DDQN	16
4.2	Confronto Finale tra i Quattro Algoritmi	17
4.3	Discussione dei Risultati	18
5	Conclusioni	19

1 Descrizione del Problema

1.1 Contesto: ADAS e Guida Autonoma

I sistemi ADAS (*Advanced Driver-Assistance Systems*) nascono con l'obiettivo fondamentale di mitigare o rimuovere la componente dell'errore umano durante la guida, combinando una suite coordinata di sensori, interfacce e processori per assistere il conducente attraverso logiche di sicurezza attive o passive.

L'evoluzione di tali dispositivi storicamente passata da sistemi di regolazione longitudinale come l'ACC (Adaptive Cruise Control) o di sicurezza obbligatori come l'ABS converge oggi verso la frontiera della Guida Autonoma (Autonomous Driving, AD), un settore interdisciplinare all'intersezione tra intelligenza artificiale, robotica e sistemi automotive.

La classificazione formale di questi sistemi è regolata dallo standard internazionale SAE J3016, il quale mappa l'automazione su sei livelli incrementali, suddivisi in due macro-gruppi descrittivi:

1. Supporto al guidatore (Livelli 0-2): Il conducente umano mantiene la responsabilità legale e il controllo costante della supervisione dinamica.

- *Livello 0:* Nessuna automazione; il sistema fornisce solo avvisi momentanei (es. Blind Spot Monitoring) o interventi di emergenza transitori.
- *Livello 1:* L'automazione controlla singolarmente il movimento longitudinale (es. ACC) o quello laterale (es. Lane Centering), ma mai entrambi contemporaneamente.
- *Livello 2:* Gestione simultanea del controllo laterale e longitudinale (es. combinazione di ACC e Lane Centering), richiedendo comunque la costante attenzione del guidatore.

2. Guida Automatizzata (Livelli 3-5): Il sistema informatico esegue l'intero compito di guida dinamica all'interno di specifici domini di progettazione operativa (ODD - Operational Design Domain).

- *Livello 3:* Automazione condizionata; il veicolo guida autonomamente in scenari circoscritti, ma richiede un intervento tempestivo del conducente (Fallback) qualora si esca dai limiti dell'ODD.
- *Livello 4:* Alta automazione; il sistema gestisce autonomamente anche le situazioni di emergenza e il Fallback all'interno del proprio ODD, consentendo al veicolo di arrestarsi in sicurezza senza input umani.
- *Livello 5:* Automazione completa e ubiquitaria, priva di vincoli geometrici, ambientali o di ODD.

Sotto il profilo ingegneristico, lo sviluppo di agenti capaci di percepire l'ambiente, pianificare traiettorie sicure e agire sui veicoli tramite tecnologie drive-by-wire si biforca in due principali filosofie software: le architetture dette modulari, che sono suddivise rigidamente in blocchi sequenziali di percezione, localizzazione, pianificazione e controllo e i paradigmi End-to-End basati interamente sui dati.

In quest'ultimo scenario, un approccio metodologico di spicco è rappresentato dal Reinforcement Learning (RL). A differenza dell'apprendimento supervisionato dove c'è la necessità di disporre di imponenti dataset preventivamente etichettati dall'uomo e soffre la gestione dei rari eventi imprevedibili, detti problemi di long-tail, l'RL permette a un agente di apprendere una politica di guida ottimale (policy) per tentativi ed errori. Formalizzato matematicamente come un Processo Decisionale di Markov (MDP), l'agente interagisce dinamicamente con l'ambiente stradale, esplorando lo spazio degli stati e delle azioni per massimizzare un segnale di feedback cumulativo nel tempo (reward function). Ciò consente di scoprire autonomamente strategie di controllo e pianificazione flessibili a differenti contesti operativi (ODD).

1.2 L'Ambiente CarRacing-v3

CarRacing-v3 è un ambiente di simulazione per il controllo ottimo e il Reinforcement Learning fornito dalla libreria **Gymnasium**. Esso implementa una pista da corsa generata proceduralmente a ogni episodio, che l'agente deve percorrere nel minor tempo possibile.

Spazio delle Osservazioni (Stato). Ad ogni passo temporale, l'agente riceve in input un'immagine RGB di dimensione $96 \times 96 \times 3$ pixel. Questa matrice rappresenta una vista dall'alto incentrata sul veicolo, mostrando la porzione di pista circostante e i dettagli nella parte inferiore.

Spazio delle Azioni. Impostando il parametro `continuous=False`, l'ambiente viene configurato con uno spazio delle azioni discreto di tipo `Discrete(5)`, il quale mette a disposizione dell'agente 5 azioni mutuamente esclusive, descritte nella Tabella 1.

Indice	Azione
0	Nessuna azione (<i>idle</i>)
1	Sterzo a sinistra
2	Sterzo a destra
3	Acceleratore
4	Freno

Tabella 1: Spazio delle azioni discrete in CarRacing-v3.

Funzione di Reward. La struttura del segnale di rinforzo è progettata per premiare la progressione e penalizzare l'inattività:

- **Premio di avanzamento:** $+1000/N$ per ogni elemento (*tile*) di pista visitato per la prima volta, dove N rappresenta il numero totale di tasselli che compongono il tracciato generato.
- **Penalità temporale:** -0.1 ad ogni singolo frame trascorso, introdotto per forzare l'agente a completare il percorso nel minor tempo possibile.

Di conseguenza, il reward teorico massimo è pari a 1000 meno la penalità accumulata nei frame necessari a completare il tracciato, assestandosi generalmente in un range compreso tra 900 e 950 per una guida ottimale e completa senza errori [8].

Condizioni di Terminazione e Troncamento. Un episodio viene dichiarato concluso (**terminated**) nel momento in cui l'agente riesce a visitare l'ultimo dei tile della pista. Viene invece troncato (**truncated**) qualora il veicolo si allontani eccessivamente dal tracciato uscendo dalla mappa, oppure al raggiungimento del limite massimo di 1000 passi operativi (limite introdotto tramite il wrapper TimeLimit di Gymnasium.).

1.3 Formulazione come Processo Decisionale di Markov

Il task è formalizzato come un **Processo Decisionale di Markov** (MDP) definito dalla tupla $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$: immagini $96 \times 96 \times 3$ dopo preprocessing
- $\mathcal{A} = \{0, 1, 2, 3, 4\}$: le 5 azioni discrete
- $\mathcal{P}$: dinamica determinata dal simulatore fisico
- $\mathcal{R}$: funzione di reward descritta sopra
- $\gamma = 0.99$: fattore di sconto temporale

L'obiettivo fondamentale dell'apprendimento è determinare una **policy** ottimale $\pi(a|s)$ — ovvero una mappatura stocastica o deterministica dagli stati alle azioni, in grado di massimizzare il valore atteso del ritorno cumulato scontato nel tempo, definito dalla funzione obiettivo:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t R(s_t, a_t) \right] \quad (1)$$

dove $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$ rappresenta la traiettoria (o episodio) generata dall'interazione continua tra la policy dell'agente e le transizioni dell'ambiente fino al tempo di terminazione o troncamento T .

2 Fondamenti Teorici

2.1 Q-Learning e Deep Q-Network (DQN)

2.1.1 Q-Learning

Il Q-Learning è un algoritmo di Reinforcement Learning basato sul valore (value-based) e di tipo model-free, il cui obiettivo è stimare la funzione di valore azione-stato $Q(s, a)$. Tale valore rappresenta la somma stimata dei reward futuri che l'agente può ottenere a partire dallo stato attuale s e dopo aver scelto l'azione a , ipotizzando che nei passi successivi selezioni sempre le azioni ottimali.

L'algoritmo opera in modalità off-policy e aggiorna iterativamente i propri valori sfruttando l'equazione di ottimalità di Bellman. La regola di aggiornamento basata sulla differenza temporale (Temporal Difference) è definita come:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (2)$$

dove $\alpha \in (0, 1]$ rappresenta il tasso di apprendimento (learning rate), r è la ricompensa immediata fornita dall'ambiente, γ è il fattore di sconto e s' definisce lo stato successivo ottenuto a seguito della transizione. Il termine racchiuso tra parentesi quadre costituisce il cosiddetto TD Error, ovvero lo scostamento tra la stima attuale e il target di Bellman aggiornato ($r + \gamma \max Q$).

Nella sua formulazione classica (tabellare), l'algoritmo mappa la funzione $Q(s, a)$ all'interno di una matrice discreta denominata Q-table, contenente una cella per ogni possibile combinazione stato-azione. Sebbene tale approccio garantisca la convergenza all'ottimo in spazi discreti finiti, esso mostra evidenti limiti di scalabilità a causa dell'esplosione combinatoria degli stati nel momento in cui viene applicato ad ambienti complessi. Nel caso specifico di CarRacing-v3, in cui lo spazio degli stati $\mathcal{S}$ è continuo ed è rappresentato da flussi di immagini RGB, la memorizzazione di una tabella risulta computazionalmente impossibile. Questo limite introduce la necessità di estendere l'algoritmo combinandolo con approssimatori funzionali non lineari basati su reti neurali profonde, dando origine alle architetture DQN e DDQN.

2.1.2 Deep Q-Network (DQN)

Nel momento in cui si opera con spazi degli stati ad alta dimensionalità o continui, come nel caso delle immagini RGB di CarRacing-v3, la classica funzione di valore non

può essere mappata tramite una struttura tabellare. L'algoritmo Deep Q-Network (DQN) [1] supera questo limite sostituendo la matrice discreta con una rete neurale profonda $Q(s, a; \theta)$, dove θ rappresenta l'insieme dei parametri o pesi della rete.

Per contrastare l'instabilità intrinseca derivante dalla combinazione di approssimatori non lineari e Reinforcement Learning, l'architettura introduce due innovazioni metodologiche fondamentali:

1. **Experience Replay:** Durante l'interazione con l'ambiente, le transizioni sperimentate dall'agente sotto forma di tuple $e_t = (s, a, r, s', d)$ — dove d è un flag booleano che indica la terminazione dell'episodio — e vengono memorizzate all'interno di un buffer di memoria circolare $\mathcal{D}$. In fase di ottimizzazione, i minibatch di addestramento vengono campionati in modo uniformemente casuale da questo archivio. Questa procedura rompe la correlazione temporale sequenziale dei dati e garantisce il soddisfacimento del vincolo di indipendenza e identica distribuzione richiesto dagli algoritmi di ottimizzazione basati sul gradiente.
2. **Target Network:** L'addestramento prevede l'impiego di due reti neurali distinte ma architetturelmente identiche. La rete online (definita dai parametri θ) viene aggiornata ad ogni passo temporale. Al contrario, la rete target (definita dai parametri θ^-) mantiene i pesi congelati e viene sincronizzata periodicamente copiando i valori da θ . La rete target viene utilizzata esclusivamente per calcolare il target di Bellman y , risolvendo il problema del moving target (l'instabilità dovuta al fatto che la rete insegue un obiettivo che cambia ad ogni step) e stabilizzando la convergenza:

$$y = r + (1 - d)\gamma \max_{a'} Q(s', a'; \theta^-) \quad (3)$$

La funzione di costo minimizzata tramite retro-propagazione dell'errore è la Smooth L1 Loss (o Huber Loss), scelta per la sua robustezza nei confronti degli outlier rispetto alla classica loss quadratica media (MSE):

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} [\text{Huber}(y - Q(s, a; \theta))] \quad (4)$$

dove l'argomento $y - Q(s, a; \theta)$ rappresenta il TD Error neurale.

Il bilanciamento tra l'esplorazione di nuove strategie e lo sfruttamento della conoscenza pregressa viene gestito tramite una politica ε -greedy. Con probabilità ε l'agente seleziona un'azione casuale all'interno dello spazio d'azione $\mathcal{A}$, mentre con probabilità $1 - \varepsilon$ adotta una logica greedy, scegliendo il comando associato al massimo valore Q stimato. Per favorire un'ampia esplorazione iniziale, il parametro ε

viene impostato a 1.0 e fatto decadere linearmente fino a un valore minimo nel corso delle epoche di addestramento.

2.1.3 Double DQN (DDQN)

L'architettura DQN classica tende intrinsecamente a sovrastimare i valori della funzione di valore (Q -value) a causa dell'operatore di massimo presente nel target di Bellman. Questo fenomeno si verifica perché la medesima rete neurale viene utilizzata sia per selezionare l'azione migliore nello stato successivo s' , sia per valutarne il relativo valore.

L'algoritmo Double DQN (DDQN) [2] risolve questo problema disaccoppiando nettamente i due ruoli e distribuendoli tra le due reti già presenti nell'architettura (la rete online e la rete target):

- **Selezione dell'azione:** La rete online (definita dai parametri θ) determina quale sia l'azione ottimale a^* da intraprendere nello stato successivo s' .
- **Valutazione dell'azione:** La rete target (definita dai parametri θ^-) stima il valore effettivo associato a tale azione a^* .

Il target di Bellman modificato viene formalizzato attraverso il seguente sistema di equazioni:

$$a^* = \arg \max_{a'} Q(s', a'; \theta), \quad y = r + (1 - d)\gamma Q(s', a^*; \theta^-) \quad (5)$$

Dal punto di vista implementativo, la variazione rispetto a DQN si riduce a poche linee di codice all'interno del ciclo di calcolo della Loss. Ciononostante, l'impatto sul processo di apprendimento è altamente significativo: la rimozione del bias di sovrastima garantisce una convergenza molto più stabile e traiettorie di guida con stime di valore coerenti con i reward reali ricevuti dall'ambiente.

2.2 Proximal Policy Optimization (PPO)

Il **Proximal Policy Optimization** (PPO) [3] appartiene alla famiglia dei metodi a gradiente della politica (policy gradient). A differenza delle architetture DQN e DDQN, che stimano il valore delle azioni per poi derivare una politica astratta, PPO ottimizza direttamente una funzione di policy parametrizzata $\pi(a|s; \theta)$, la quale mappa gli stati direttamente in una distribuzione di probabilità sulle azioni possibili.

2.2.1 Funzione di Vantaggio e GAE

Per aggiornare i pesi della politica, l'algoritmo stima l'efficacia dell'azione intrapresa rispetto alla media tramite l'Advantage Function A_t . In particolare, come nel pre-

sente progetto si utilizza la tecnica GAE (Generalized Advantage Estimation) [4], la quale calcola il vantaggio come somma pesata dei reward futuri scontati:

$$A_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l [r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l})] \quad (6)$$

Il parametro $\lambda \in [0, 1]$ agisce come un regolatore del trade-off tra distorsione (bias) e varianza del gradiente, stabilizzando l'orientamento degli aggiornamenti.

2.2.2 Clipped Surrogate Loss

La caratteristica distintiva di PPO è l'introduzione di un meccanismo di vincolo (clipping) sul rapporto di probabilità tra la nuova politica e la vecchia politica (policy ratio):

$$r_t(\theta) = \frac{\pi(a_t|s_t; \theta)}{\pi(a_t|s_t; \theta_{\text{old}})} \quad (7)$$

Per evitare che un singolo aggiornamento modifichi la policy in modo troppo drastico (causando il collasso delle prestazioni tipico dei metodi on-policy classici), PPO massimizza una funzione obiettivo surrogata e ritagliata (Clipped Surrogate Loss):

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) A_t)] \quad (8)$$

Il parametro di clipping viene impostato al valore standard $\varepsilon = 0.2$, forzando il rapporto $r_t(\theta)$ a rimanere confinato nell'intervallo $[0.8, 1.2]$ se il vantaggio spinge verso variazioni eccessive.

2.2.3 Loss Totale Actor-Critic

Il sistema adotta un'architettura Actor-Critic implementata tramite una rete neurale condivisa con due teste di output distinte: l'Attore (Actor), che approssima la policy $\pi(a|s)$, e il Critico (Critic), che stima la funzione di valore dello stato $V(s)$. La rete viene ottimizzata end-to-end minimizzando una funzione di loss totale:

$$\mathcal{L}^{\text{TOTAL}} = \mathcal{L}^{\text{CLIP}} - c_1 \mathcal{L}^{\text{VF}} + c_2 \mathcal{H}[\pi] \quad (9)$$

dove $\mathcal{L}^{\text{VF}}$ rappresenta l'errore quadratico medio (MSE) sulla stima del valore del Critico, mentre $\mathcal{H}[\pi]$ introduce un bonus di entropia sulla policy per incentivare l'esplorazione ed evitare la convergenza prematura verso minimi locali. I coefficienti sono configurati con i valori standard $c_1 = 0.5$ e c_2 bilanciato in base alla fase del training.

2.2.4 Azioni Discrete e Distribuzione Categorical

La scelta di configurare CarRacing-v3 in modalità discreta consente di modellare l'output dell'Attore tramite una distribuzione probabilistica di tipo Categorical (basata sulle 5 azioni disponibili). Questa scelta si rivela cruciale per la stabilità del sistema. Nelle varianti a controllo continuo (basate su distribuzioni Gaussiane), il parametro della varianza $\log \sigma$ può decrescere senza vincoli matematici rigidi, portando a picchi di gradiente e instabilità. Al contrario, l'utilizzo di una distribuzione Categorical confina matematicamente l'entropia massima al valore costante $\ln(5)$, offrendo un tetto strutturale che protegge l'algoritmo da divergenze numeriche garantendo un apprendimento robusto.

2.3 Confronto tra i Paradigmi

Caratteristica	DQN / DDQN	PPO
Tipo	Off-policy (value-based)	On-policy (policy gradient)
Ottimizza	Q-function	Policy π direttamente
Experience replay	Sì	No
Sample efficiency	Alta	Bassa
Stabilità	Target network	Clipping
Env. paralleli	Non necessari	Raccomandati

Tabella 2: Confronto tra i paradigmi DQN/DDQN e PPO.

3 Implementazione

3.1 Pipeline di Preprocessing

Per l'implementazione siamo partiti dall'input grezzo fornito da CarRacing-v3, cioè un'immagine RGB di dimensione $96 \times 96 \times 3$ pixel aggiornata ad ogni singolo frame. Operare direttamente su questa rappresentazione sarebbe computazionalmente proibitivo e introdurrebbe ridondanza temporale eccessiva, poiché frame consecutivi differiscono minimamente per questo sono stati applicati in cascata i seguenti wrapper di Gymnasium:

1. **SkipFrame** ($k = 4$): l'agente sceglie un'azione ogni 4 frame, ripetendola automaticamente negli step intermedi. Questo riduce la durata effettiva di un episodio di un fattore 4, velocizzando il training di circa $4\times$ senza perdita significativa di reattività.

2. **GrayscaleObservation**: l'immagine RGB viene convertita in scala di grigi, riducendo i canali da 3 a 1. L'informazione cromatica non è rilevante per il controllo del veicolo.
3. **ResizeObservation** (84×84): ridimensionamento al formato standard adottato nella letteratura DQN per task visivi.
4. **FrameStackObservation** ($n = 4$): i 4 frame più recenti vengono impilati lungo la dimensione dei canali, producendo un tensore di forma $(4, 84, 84)$. Questo conferisce all'agente una *memoria visiva a breve termine*, rendendo percepibili velocità e direzione del moto.

Questa pipeline è condivisa da DQN, DDQN e PPO custom. Il modello SB3-PPO adotta invece una variante modificata.

3.2 Architettura della Rete Neurale

L'estrattore di feature visive è una CNN ispirata all'architettura NIPS 2013 di DeepMind [1]. La versione baseline (2 strati conv) riceve in input il tensore $(4, 84, 84)$:

$$\begin{aligned} \text{Conv}(4, 16, k=8, s=4) \rightarrow \text{ReLU} \rightarrow \text{Conv}(16, 32, k=4, s=2) \rightarrow \text{ReLU} \\ \rightarrow \text{Flatten} \rightarrow \text{Linear}(2592, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 5) \end{aligned} \quad (10)$$

Nel corso degli esperimenti questa architettura è stata estesa aggiungendo un terzo strato convoluzionale (3-conv), in linea con la versione Nature 2015 [1]:

$$\begin{aligned} \text{Conv}(4, 16, k=8, s=4) \rightarrow \text{ReLU} \rightarrow \text{Conv}(16, 32, k=4, s=2) \rightarrow \text{ReLU} \\ \rightarrow \text{Flatten} \rightarrow \text{Linear}(2592, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 5) \end{aligned} \quad (11)$$

Il terzo strato consente di estrarre feature gerarchiche di livello superiore, come la curvatura della pista in lontananza.

3.3 Evoluzione Iterativa di DQN e DDQN

L'implementazione di DQN e DDQN è sviluppata all'interno dello stesso modulo, che espone un singolo parametro booleano per selezionare l'algoritmo. Le varianti sono state addestrate per circa 740k step.

3.3.1 Prima Iterazione: Baseline e Stabilizzazione

Il primo training ha utilizzato l'architettura a 2 strati convoluzionali con $\gamma = 0.95$, buffer di 50k transizioni e sincronizzazione della target network ogni 5k step. Durante l'addestramento è emerso un problema critico: la loss ha raggiunto valori fino a 1.4, con i gradienti che crescevano in modo incontrollato (gradient explosion), destabilizzando i pesi della rete.

Sono stati apportati quattro interventi correttivi rispetto alla configurazione iniziale:

1. **Buffer** 50k $\rightarrow$ 200k: la memoria più ampia garantisce maggiore diversità nelle transizioni campionate, riducendo la correlazione tra minibatch consecutivi.
2. **Discount factor** $\gamma 0.95 \rightarrow 0.99$: l'orizzonte temporale effettivo passa da ~ 20 a ~ 100 step, permettendo all'agente di pianificare le azioni necessarie per affrontare curve che si trovano ancora a distanza.
3. **Sincronizzazione target** 5k $\rightarrow$ 10k step: la rete target rimane ferma più a lungo, stabilizzando il bersaglio di Bellman e riducendo le oscillazioni del training.
4. **Gradient clipping** ($\text{max_norm} = 10$): un limite superiore sul modulo del gradiente impedisce fisicamente le esplosioni di peso, il fix più impattante tra i quattro.

3.3.2 Secondo Miglioramento: Terzo Strato Convoluzionale

Motivati dall'architettura Nature DQN [1], è stato aggiunto un terzo strato convoluzionale. L'effetto principale è una drastica riduzione della varianza per episodio: il DQN passa da una deviazione standard di 54 circa a 17, indicando una policy molto più robusta su tracciati proceduralmente diversi.

3.3.3 Terzo Miglioramento: Prioritized Experience Replay

Il campionamento uniforme dal replay buffer tratta tutte le transizioni come equiprobabili. Nel contesto di CarRacing-v3, la maggior parte dei frame corrisponde a guida dritta (TD-error basso), mentre le transizioni critiche — uscite di pista, curve strette — sono più rare ma molto più informative (TD-error alto).

Il Prioritized Experience Replay (PER) [5] assegna a ogni transizione una priorità proporzionale al TD-error, campionando con più frequenza le esperienze più sorprendenti per la rete. I risultati sperimentali hanno mostrato effetti incoerenti: il PER migliora la convergenza nelle prime fasi di training ma non garantisce un vantaggio

sistematico sul reward medio finale, suggerendo che a budget di step fisso possa sbilanciare il training verso esperienze critiche rare a scapito del consolidamento della guida ordinaria.

3.3.4 Quarto Esperimento: Spazio delle Azioni Esteso a 9 Comandi

Con le 5 azioni discrete standard l'agente non può eseguire sterzo e accelerazione simultaneamente, il che limita la fluidità in curva. È stato quindi sperimentato uno spazio delle azioni a 9 comandi, costruito come prodotto cartesiano:

$$\{\text{nessuna azione, Gas, Freno}\} \times \{\text{nessuna azione, Sinistra, Destra}\}$$

I risultati sperimentali hanno smentito l'ipotesi di miglioramento. Il DQN con 9 azioni ha registrato una media di 565.7 con deviazione standard di 160.7, significativamente inferiore alla variante a 5 azioni. Il DDQN ha mostrato un comportamento analogamente instabile (media 758.8, std 213.7). La causa principale è la maggiore difficoltà di esplorazione: uno spazio delle azioni più ampio richiede molti più step per convergere a una policy stabile, dato il budget di training fisso a $\sim 740k$ step, che richiede comunque un tempo di training di circa 15 ore, per cui test su più step per il raggiungimento di risultati significativi sarebbero stati su periodi particolarmente lunghi e dispendiosi. Le 9 azioni sono quindi state scartate.

3.3.5 Versione Finale

A seguito dell'analisi comparativa, l'architettura finale adottata è la variante *3conv+PER* con 5 azioni discrete, gradient clipping e iperparametri consolidati. La Tabella 3 riassume la configurazione definitiva.

Categoria	Parametro	Valore
Rete neurale	Architettura CNN	3 strati conv (Nature DQN)
	Learning rate	2×10^{-4}
	Gradient clipping	max_norm = 10
Esplorazione	ε_0 / $\varepsilon_{\min}$ / decay	1.0 / 0.05 / 0.9999925
Replay buffer	Dimensione	300 000 transizioni
	Campionamento	Prioritized (PER)
Training	Batch size	32
	Aggiornamento rete	ogni 4 step
	Sincronizzazione target	ogni 10 000 step
	Discount factor γ	0.99
Ambiente	SkipFrame	4
	Frame stack	4

Tabella 3: Iperparametri finali per DQN e DDQN.

3.4 PPO con Implementazione Custom

L’implementazione custom di PPO adotta un’architettura Actor-Critic con CNN condivisa. La rete estrattrice di feature è identica alla variante 3-conv; ma su di essa si innestano due moduli distinti:

- **Attore (Actor):** Uno strato lineare fully-connected preposto alla generazione dei logit non normalizzati, i quali alimentano la distribuzione probabilistica di tipo Categorical sulle 5 azioni discrete.
- **Critico (Critic):** Uno strato lineare fully-connected adibito alla stima del valore scalare atteso dello stato corrente $V(s)$.

Il training raccoglie i rollout di lunghezza fissa su un ambiente singolo e calcola il vantaggio tramite il GAE, ed esegue K epoche di aggiornamento su minibatch con la Clipped Surrogate Loss.

Il modello finale qui è stato addestrato per circa 1M di step [10] poichè teoricamente avrebbe dovuto portare a dei risultati incoraggianti, cosa invece abbastanza discutibile.

Parametro	Valore
Discount factor γ	0.99
GAE λ	0.95
Clipping ε	0.2
Coefficiente critico c_1	0.5
Step totali	$\sim 1\text{M}$
SkipFrame / Frame stack	4 / 4

Tabella 4: Iperparametri PPO custom.

3.5 SB3-PPO con Ambienti Paralleli

Il quarto algoritmo valutato è PPO tramite la libreria Stable-Baselines3 [6], che offre un’implementazione ottimizzata con supporto nativo per la parallelizzazione degli ambienti.

Pipeline di osservazione modificata. A differenza degli altri tre algoritmi, SB3-PPO non applica la conversione in scala di grigi. Il preprocessing si riduce a SkipFrame e ResizeObservation (84×84), ottenendo osservazioni RGB di forma $(84, 84, 3)$. Il wrapper VecFrameStack impila poi 4 frame lungo la dimensione dei canali, producendo un vettore $(84, 84, 12)$ come input alla NatureCNN interna di SB3.

Questa scelta è stata imposta da un vincolo tecnico: la combinazione di GrayscaleObservation e VecFrameStack produceva vettori di forma errata $(84, 336)$ invece di $(84, 84, 4)$, causando un’incompatibilità con la NatureCNN di SB3. La rimozione del passaggio grayscale ha risolto il problema mantenendo le stesse capacità percettive.

Parallelizzazione. Il training utilizza 8 ambienti paralleli (SubprocVecEnv), che raccolgono simultaneamente 8 traiettorie distinte ad ogni rollout. Questo aumenta la diversità dei dati e riduce la correlazione temporale tra campioni on-policy, compensando parzialmente il principale svantaggio dei metodi policy gradient rispetto agli approcci off-policy.

Parametro	Valore
Step totali di training	2 000 000
Ambienti paralleli	8 (SubprocVecEnv)
Step per rollout (n_steps)	512
Batch size	128
Learning rate	10^{-4}
Coefficiente entropia	0.0
Discount factor γ	0.99
Clipping ε	0.2
Policy	NatureCNN

Tabella 5: Iperparametri SB3-PPO.

4 Risultati e Analisi

4.1 Ablation Study: Evoluzione delle Varianti DQN/DDQN

La Tabella 6 raccoglie i risultati della valutazione sistematica di tutte le varianti sviluppate, ciascuna testata su 10 episodi con policy completamente greedy ($\varepsilon = 0$).

Variante	Media	Std	Max
DQN baseline	852.1	54.2	927.3
DQN 3conv	885.8	16.9	918.2
DQN 3conv + PER	866.1	53.1	923.2
DQN 3conv + PER + 9a	733.7	108.0	858.7
DDQN baseline	818.7	68.0	892.6
DDQN 3conv	873.4	31.2	912.1
DDQN 3conv + PER	819.7	60.6	884.0
DDQN 3conv + PER + 9a	797.3	182.2	936.2

Tabella 6: Ablation study: reward medio su 10 episodi per ciascuna variante DQN/DDQN.

I dati evidenziano tre tendenze principali:

- **Il terzo strato convoluzionale è il miglioramento più consistente.** Il passaggio da 2 a 3 strati conv riduce drasticamente la deviazione standard del DQN da 54.2 a 16.9 e del DDQN da 68.0 a 31.2, segnalando una policy più

robusta rispetto alla variabilità dei tracciati procedurali. L'effetto sulla media è anch'esso positivo in entrambi i casi.

- **Il PER non garantisce un miglioramento sistematico.** Per il DQN il PER riduce la media rispetto alla sola variante 3-conv (da 885.8 a 866.1); per il DDQN i risultati sono analoghi. Il PER migliora la convergenza nelle fasi iniziali ma, a budget di step fisso, può sbilanciare il training verso transizioni critiche rare a scapito del consolidamento delle skill di guida ordinaria.
- **Le 9 azioni peggiorano le performance.** L'espansione dello spazio delle azioni introduce una complessità di esplorazione che non è compensabile con 740k step di training: la deviazione standard aumenta in modo significativo (fino a 182.2 per DDQN), indicando un apprendimento non convergente nei tempi disponibili.

4.2 Confronto Finale tra i Quattro Algoritmi

Il confronto finale è condotto sui modelli definitivi di ciascun algoritmo: i modelli di riferimento pre-addestrati ($\sim 740k$ step) per DQN e DDQN, il modello custom addestrato autonomamente ($\sim 1M$ step) per PPO, e il modello SB3-PPO (2M step). Tutti e quattro gli algoritmi sono valutati su **30 episodi** con policy greedy su tracciati generati proceduralmente [9]., garantendo uniformità statistica nel confronto.

Algoritmo	Media	Std	Max	N episodi
DQN	854.2	85.2	933.1	30
DDQN	852.6	91.5	921.4	30
PPO (custom)	552.9	112.7	—	30
SB3-PPO	679.3	174.1	—	30

Tabella 7: Confronto finale tra i quattro algoritmi su CarRacing-v3 (30 episodi ciascuno).

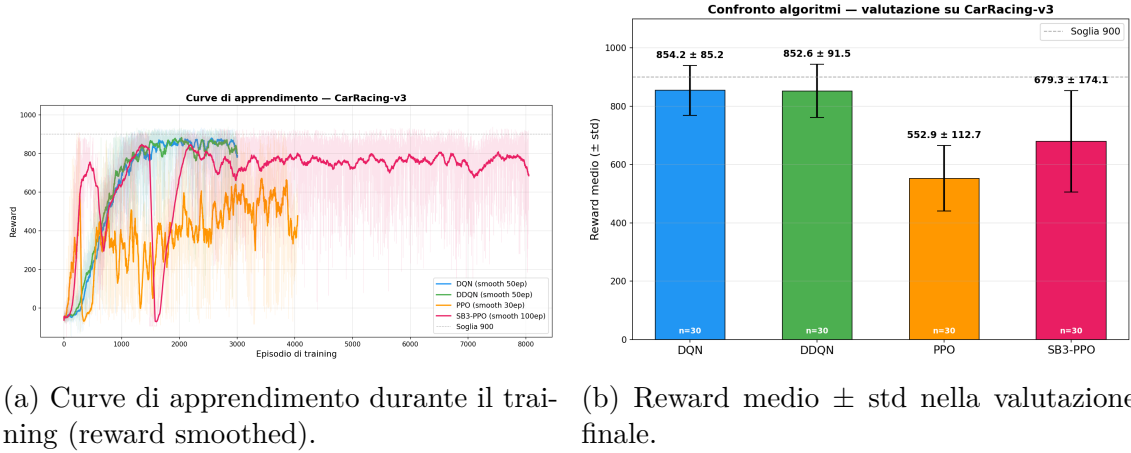


Figura 1: Confronto visuale tra i quattro algoritmi su CarRacing-v3.

4.3 Discussione dei Risultati

DQN e DDQN sono equivalenti su campione statisticamente robusto. Estendendo la valutazione a 30 episodi, i due algoritmi convergono a reward medi quasi identici (DQN 854.2, DDQN 852.6) con deviazioni standard comparabili (85.2 vs 91.5). Questo risultato è in apparente contraddizione con il vantaggio teorico atteso da DDQN nella rimozione del bias di sovrastima. Una valutazione preliminare su soli 10 episodi aveva invece mostrato una differenza molto più marcata (DQN 790.3 ± 143.2 , DDQN 850.1 ± 37.8), rivelatasi poi un artefatto statistico: DQN aveva incontrato per caso alcune piste particolarmente difficili, producendo episodi con reward molto bassi (588, 598) che gonfiavano artificialmente la deviazione standard. Con 30 episodi, entrambi gli algoritmi mostrano episodi problematici analoghi, e le differenze si livellano. Questo sottolinea l'importanza di un campione di valutazione sufficientemente ampio in ambienti proceduralmente casuali come CarRacing-v3.

I metodi off-policy superano quelli on-policy. DQN e DDQN raggiungono reward medi significativamente più alti di PPO custom e SB3-PPO, nonostante abbiano ricevuto meno step di training (740k vs 1M e 2M). La spiegazione risiede nella maggiore *sample efficiency* dei metodi off-policy: grazie al replay buffer, ogni transizione può essere riutilizzata più volte nel calcolo del gradiente. I metodi on-policy come PPO scartano ogni batch di esperienza dopo un singolo aggiornamento, richiedendo molti più step per convergere a livelli comparabili.

La parallelizzazione non compensa il divario on/off-policy. SB3-PPO, pur disponendo di 8 ambienti paralleli e 2M step, non raggiunge le performance di DQN e DDQN. La parallelizzazione migliora la velocità di raccolta dati ma non la *sample efficiency* intrinseca dell'algoritmo. SB3-PPO supera tuttavia il PPO custom (679.3

vs 552.9), confermando il valore pratico della parallelizzazione all'interno della stessa famiglia algoritmica.

5 Conclusioni

Questo elaborato ha confrontato quattro implementazioni di algoritmi di Reinforcement Learning su CarRacing-v3: DQN, DDQN, PPO custom e SB3-PPO. I risultati principali sono:

- **DQN e DDQN raggiungono performance equivalenti** su 30 episodi (854.2 vs 852.6): il vantaggio teorico di DDQN nella riduzione del bias di sovrastima non produce un miglioramento misurabile del reward medio su questo task a parità di step di training.
- **I metodi off-policy (DQN/DDQN) sono più efficienti** dei metodi on-policy (PPO) su questo task, raggiungendo reward più alti con meno step di training grazie al replay buffer.
- **Il terzo strato convoluzionale** (architettura Nature DQN, Mnih et al. 2015) è il singolo miglioramento più impattante nell'ablation study, riducendo consistentemente la varianza per episodio in tutte le varianti testate.
- **Le 9 azioni combinate** hanno peggiorato le performance: la maggiore espressività dello spazio delle azioni non è sfruttabile con il budget di training disponibile.
- **Il PER ha effetti incoerenti**: utile nelle prime fasi del training, non garantisce vantaggi sistematici sul reward finale a budget fisso.

Come sviluppo futuro, sarebbe interessante approfondire l'applicazione dell'algoritmo SAC (Soft Actor-Critic) in modalità azione continua, che combina la sample efficiency dei metodi off-policy con la stabilità dei policy gradient tramite il principio di entropia massima. Tentativi presenti in letteratura informale hanno già applicato SAC a CarRacing-v2 senza raggiungere risultati ottimali [7], suggerendo che la complessità dello spazio delle azioni continue rimanga una sfida aperta per gli algoritmi attuali. Un confronto sistematico a parità di step e di episodi di valutazione potrebbe chiarire ulteriormente i vantaggi relativi dei due paradigmi.

Riferimenti bibliografici

- [1] Mnih, V. et al. (2015). *Human-level control through deep reinforcement learning*. Nature, 518(7540), 529–533.

- [2] Van Hasselt, H., Guez, A., & Silver, D. (2016). *Deep Reinforcement Learning with Double Q-Learning*. AAAI Conference on Artificial Intelligence.
- [3] Schulman, J. et al. (2017). *Proximal Policy Optimization Algorithms*. arXiv:1707.06347.
- [4] Schulman, J. et al. (2015). *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. arXiv:1506.02438.
- [5] Schaul, T. et al. (2015). *Prioritized Experience Replay*. arXiv:1511.05952.
- [6] Raffin, A. et al. (2021). *Stable-Baselines3: Reliable Reinforcement Learning Implementations*. Journal of Machine Learning Research, 22(268), 1–8.
- [7] Finding Theta (2024). *Solving Gymnasium’s Car Racing with Reinforcement Learning*. <https://www.findingtheta.com/blog/solving-gymnasiums-car-racing-with-reinforcement-learning>
- [8] NotAnyMike (2019). *Solving Car Racing*. <https://notanymike.github.io/Solving-CarRacing/>
- [9] Sandke, K. (2024). *AICarRacing: Reinforcement Learning on OpenAI Gym CarRacing*. <https://github.com/KSandke/AICarRacing>
- [10] Cholula, F. (2024). *PPO-LSTM model trained on CarRacing-v3*. Hugging Face Model Hub. https://huggingface.co/frankcholula/ppo_lstm-CarRacing-v3